ADVANCED RAG SERIES | PART 4 OF 5

Agentic, Self-Reflective and Graph RAG

Giving retrieval judgement, tools and structure

Covers: Self-RAG, CRAG, Adaptive RAG, FLARE, LangGraph control flow, ReAct tool agents, guardrails, knowledge graphs, GraphRAG and architecture choice.

A practitioner's guide: concepts, trade-offs, code and checklists

Contents

1. From Fixed Pipelines to Control Flow 3

1.1 The spectrum of RAG architectures . . . 3

2. Reflection Patterns from the Research Literature 3

2.1 Self-RAG . . . 3

2.2 Corrective RAG (CRAG) . . . 4

2.3 Adaptive RAG . . . 4

2.4 Active retrieval: FLARE . . . 4

3. Building Control Flow with LangGraph 4

3.1 The three building blocks . . . 5

3.2 A corrective RAG graph . . . 5

4. Tool-Using RAG Agents 6

4.1 Retrieval as a tool . . . 7

4.2 Writing good tool descriptions . . . 7

4.3 Planning, reflection and multi-agent designs . . . 8

4.4 Memory . . . 8

5. Guardrails for Loops and Autonomy 8

6. Knowledge Graphs and GraphRAG 8

6.1 Two kinds of graph-based RAG . . . 9

6.2 The GraphRAG indexing pipeline . . . 9

6.3 Local and global search . . . 9

6.4 A minimal graph construction sketch . . . 9

6.5 Text-to-Cypher and graph-enhanced vector search . . . 10

6.6 Alternatives and trade-offs . . . 10

7. Choosing the Right Architecture 11

7.1 Case study: an enterprise support assistant . . . 11

7.2 Evaluating agentic systems . . . 12

7.3 Checklist . . . 12

7.4 Summary and further reading . . . 12

1. From Fixed Pipelines to Control Flow

Everything covered so far in this series is a **pipeline**: the steps run in a fixed order for every query. Transform the query, retrieve, rerank, generate. Pipelines are predictable, fast and easy to debug, and for a large share of questions they are exactly right. But they embody a hidden assumption: that one retrieval pass, using the first formulation of the query, will return enough evidence. When that assumption fails, a pipeline has no way to notice or react. It generates an answer from whatever it was given, relevant or not.

Agentic and self-reflective RAG replaces the fixed order with **control flow**. The system inspects what it has retrieved, judges whether it is adequate, and chooses what to do next: answer, reformulate and search again, consult a different source, decompose the question, or admit that it does not know. The language model becomes a decision-maker in the loop, and not only a text generator at the end.

1.1 The spectrum of RAG architectures

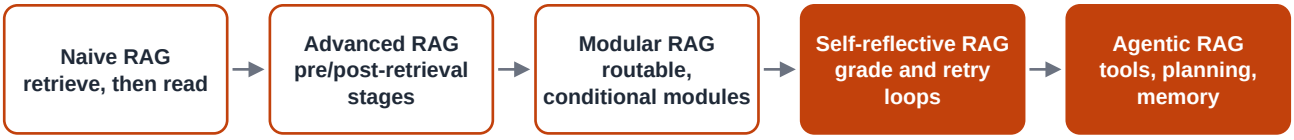


Figure 1. Increasing autonomy and cost from left to right. Choose the lowest level that meets quality needs.

Each step to the right buys flexibility on hard queries and pays for it in latency, token cost, variance and difficulty of testing. The right-hand architectures are not "better"; they are appropriate for questions where a single pass is unreliable: multi-hop reasoning, ambiguous or underspecified requests, heterogeneous data sources, and tasks that must be verified before answering.

> **KEY IDEA** Adaptivity should be proportional to difficulty. Easy questions should take the cheap path; only hard questions should trigger loops, extra tools and multiple model calls.

2. Reflection Patterns from the Research Literature

Several influential papers define the vocabulary of self-reflective RAG. Understanding them helps you design your own variants and judge vendor claims.

2.1 Self-RAG

Self-RAG (Asai et al., 2023) trains a single language model to generate special **reflection tokens** alongside text. These tokens let the model make explicit decisions during generation:

- **Retrieve**: should I retrieve now, or can I answer from my own knowledge?
- **IsRel**: is this retrieved passage relevant to the question?
- **IsSup**: is my generated segment supported by the passage?
- **IsUse**: is the overall response useful to the user?

At inference time the model generates candidate continuations for each retrieved passage, scores them using the reflection tokens, and selects the best segments. Because the critique is built into the model, no external judge is required, and the behaviour can be steered by adjusting thresholds (for example, favouring strict factual support over fluency). The practical lesson, which transfers even if you do not train such a model, is the decomposition of quality into three separately checkable properties: relevance of evidence, support of claims by evidence, and usefulness of the answer.

2.2 Corrective RAG (CRAG)

CRAG (Yan et al., 2024) adds a lightweight **retrieval evaluator** that scores how well the retrieved documents match the query and sorts the outcome into three cases:

- **Correct.** At least one document is confidently relevant. The system refines the knowledge: it splits the documents into small strips, filters out irrelevant strips, and recomposes the rest (a decompose-then-recompose step).
- **Incorrect.** Retrieval failed. The system discards the retrieved documents and falls back to a **web search** (after rewriting the query for search) to find new evidence.
- **Ambiguous.** The evaluator is unsure. The system combines refined internal knowledge with web results.

The key idea is a cheap quality gate between retrieval and generation, with an action attached to each verdict. It is straightforward to implement with an LLM grader or a reranker score and is the most widely copied pattern in practice.

2.3 Adaptive RAG

Adaptive RAG (Jeong et al., 2024) trains a small classifier to predict a query's complexity and route accordingly: no retrieval for simple questions the model already knows, single-step retrieval for moderate questions, and multi-step iterative retrieval for complex ones. This is the router from Part 1, specialised to complexity, and it captures the central economic point: pay for iteration only when needed.

2.4 Active retrieval: FLARE

FLARE (Jiang et al., 2023) retrieves **during** generation. The model drafts the next sentence; if any token has low probability (a signal of uncertainty), the draft is used as a query, relevant passages are retrieved, and the sentence is regenerated with the new evidence. Retrieval therefore happens exactly where the model is unsure. It requires access to token probabilities and adds latency in long answers, so it is mostly used for long-form generation.

Pattern	Decision it adds	Signal used	Action on failure
Self-RAG	Retrieve or not; accept segment or not	Reflection tokens from a trained model	Choose another passage or segment
CRAG	Is retrieval good enough?	Retrieval evaluator score	Web search; refine or discard documents
Adaptive RAG	How much effort does this query need?	Complexity classifier	Escalate to multi-step retrieval
FLARE	Is the model uncertain right now?	Token probabilities	Retrieve with the draft sentence as query

Table 1. Four self-reflective patterns and the decision each introduces.

3. Building Control Flow with LangGraph

To implement these patterns you need a way to express loops, branches and shared state. **LangGraph** models an application as a graph: a typed **state** object, **nodes** (functions that read the state and return updates), and **edges** (fixed or conditional transitions). Because the graph can contain cycles, it naturally supports retry loops and agents, with built-in support for persistence, streaming, human interrupts and step limits.

3.1 The three building blocks

- **State.** A typed dictionary (commonly a `TypedDict`) that flows through the graph. Each node returns a **partial** update, and the framework merges it into the state. For fields such as chat messages, a **reducer** function (for example `add_messages`) specifies how updates combine, appending new messages instead of overwriting the list.
- **Nodes.** Plain Python functions: retrieve, grade, rewrite, generate, call a tool. Each performs one well-defined step, which makes it testable in isolation.
- **Edges.** `add_edge` for fixed transitions and `add_conditional_edges` for routing functions that inspect the state and return the name of the next node.

3.2 A corrective RAG graph

The graph below implements a CRAG-style loop with grading, query rewriting, a web-search fallback, a grounding check on the generated answer and a retry counter that guarantees termination. Read the routing functions carefully: they are where the system's policy lives.

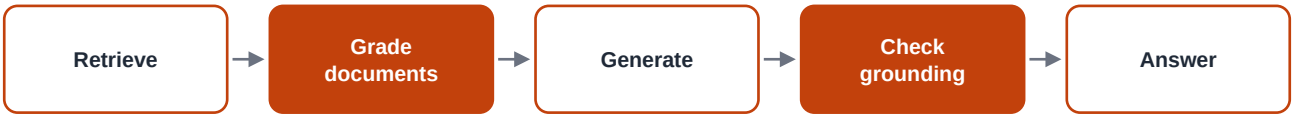


Figure 2. Happy path. Failed grading triggers rewrite and retry (or web search); failed grounding regenerates.

```
from typing import TypedDict, Literal
from langgraph.graph import StateGraph, START, END
from langchain_core.documents import Document
from pydantic import BaseModel

class RAGState(TypedDict):
    question: str
    query: str                  # current (possibly rewritten) search query
    documents: list[Document]
    answer: str
    attempts: int               # guards against infinite loops

class Grade(BaseModel):
    relevant: bool

grader = llm.with_structured_output(Grade)

def retrieve(state: RAGState):
    docs = hybrid.invoke(state["query"])
    return {"documents": docs}

def grade(state: RAGState):
    keep = []
    for d in state["documents"]:
        g = grader.invoke(f"Question: {state['question']}\nPassage: {d.page_content}\n"
                          "Is the passage relevant to the question?")
        if g.relevant:
            keep.append(d)
    return {"documents": keep}

def rewrite(state: RAGState):
    new_q = rewriter.invoke({"domain": "our product docs", "question": state["question"]})
    return {"query": new_q, "attempts": state["attempts"] + 1}

def web_search(state: RAGState):
    results = web_tool.invoke(state["question"])           # any search tool
    return {"documents": [Document(page_content=r["content"]) for r in results]}

def generate(state: RAGState):
    ctx = build_context(state["documents"])
    return {"answer": answer_chain.invoke({"context": ctx, "question": state["question"]})}
```

Listing 1. State and nodes for a corrective RAG graph (part 1 of 2).

```
class Grounded(BaseModel):
    grounded: bool

grounding_judge = llm.with_structured_output(Grounded)

def after_grade(state: RAGState) -> Literal["generate", "rewrite", "web_search"]:
    if state["documents"]:                       # at least one relevant passage survived
        return "generate"
    if state["attempts"] < 1:
        return "rewrite"                         # one reformulation attempt first
    return "web_search"                          # then fall back to the web

def after_generate(state: RAGState) -> Literal["done", "rewrite"]:
    ctx = build_context(state["documents"])
    g = grounding_judge.invoke(
        f"Sources:\n{ctx}\n\nAnswer:\n{state['answer']}\n"
        "Is every claim in the answer supported by the sources?")
    if g.grounded or state["attempts"] >= 2:
        return "done"
    return "rewrite"

builder = StateGraph(RAGState)
for name, fn in [("retrieve", retrieve), ("grade", grade), ("rewrite", rewrite),
                 ("web_search", web_search), ("generate", generate)]:
    builder.add_node(name, fn)

builder.add_edge(START, "retrieve")
builder.add_edge("retrieve", "grade")
builder.add_conditional_edges("grade", after_grade)
builder.add_edge("rewrite", "retrieve")
builder.add_edge("web_search", "generate")
builder.add_conditional_edges("generate", after_generate, {"done": END, "rewrite": "rewrite"})

app = builder.compile()
out = app.invoke({"question": "Why does error E4021 appear after the 2.3 update?",
                  "query": "", "documents": [], "answer": "", "attempts": 0})
print(out["answer"])
```

Listing 2. Routing functions and graph assembly (part 2 of 2).

Several design choices in this graph deserve emphasis. The **attempt counter** is stored in the state and checked in both routers, so the loop always terminates. Grading is performed per document with a structured boolean output, which is cheap, and which can be replaced by a reranker score if you want to avoid an LLM call. The grounding check is a **second opinion** on the generated answer, using the same sources: it catches answers that drift beyond the evidence. And when evidence is genuinely missing, the graph should end with an honest "I could not find this" answer, not an invented one; add that branch explicitly for the case that web search also returns nothing.

> **PITFALL** LLM graders are themselves fallible and add latency. Grade in parallel, batch the calls, use a small fast model, and evaluate the grader against human labels. A grader that rejects good passages can cause needless retries; one that accepts bad passages defeats the purpose.

4. Tool-Using RAG Agents

In the graph above the developer fixed the topology. In an **agent**, the model chooses at each step which tool to call, if any, observes the result and decides what to do next, until it decides it can answer. The dominant pattern is **ReAct** (Yao et al., 2022), which interleaves reasoning ("I need the warranty period for model X200") with actions (calling a search tool) and observations (the tool's result).

4.1 Retrieval as a tool

Expose each knowledge source as a tool with a clear name, a description written for the model, and a typed input schema. The description is the interface: it should state what the tool contains, when to use it, and when not to. For a multi-source assistant you might define a document-search tool, a SQL tool (Part 1, Text-to-SQL), a graph-query tool, a web-search tool and a calculator. The agent then performs routing, decomposition and iteration through its own tool choices.

```
from langchain_core.tools import tool
from langgraph.graph import StateGraph, START, MessagesState
from langgraph.prebuilt import ToolNode, tools_condition

@tool
def search_docs(query: str) -> str:
    """Search product manuals and policies. Use for how-to, troubleshooting and
    warranty questions. Input: a specific, self-contained search query."""
    docs = rerank(query, hybrid.invoke(query), top_n=5)
    return build_context([d for d, _ in docs])

@tool
def query_orders_db(question: str) -> str:
    """Answer numeric questions about orders and revenue from the SQL warehouse.
    Input: a natural-language question. Read-only."""
    return text_to_sql_chain.invoke({"question": question})

tools = [search_docs, query_orders_db]
llm_with_tools = llm.bind_tools(tools)

def assistant(state: MessagesState):
    sys = ("You are a support analyst. Use tools to gather evidence, cite sources, "
           "and say so if you cannot find the answer.")
    return {"messages": [llm_with_tools.invoke([("system", sys)] + state["messages"])]}

g = StateGraph(MessagesState)                    # MessagesState uses the add_messages reducer
g.add_node("assistant", assistant)
g.add_node("tools", ToolNode(tools))
g.add_edge(START, "assistant")
g.add_conditional_edges("assistant", tools_condition)   # tool call -> "tools", else END
g.add_edge("tools", "assistant")
agent = g.compile()

question = "Which model has the most warranty claims and what does its warranty cover?"
result = agent.invoke({"messages": [("user", question)]},
                      config={"recursion_limit": 12})
```

Listing 3. A tool-calling RAG agent. The reducer appends messages, so the model sees the full tool trace.

4.2 Writing good tool descriptions

- Name tools by what they return, not by implementation (`search_policies`, not `vector_db_query`).
- State the **scope and boundaries**: "Do not use for questions about pricing."
- Describe input format with an example; ask for self-contained queries, because the tool will not see the conversation.
- Return compact, structured results with source identifiers. Large raw dumps exhaust the context and confuse the model.
- Return **helpful errors** ("No results for model X900; known models are...") so the agent can correct itself.

4.3 Planning, reflection and multi-agent designs

For long tasks, an explicit **plan-and-execute** structure often works better than free-form ReAct: a planner produces a list of steps, an executor carries them out (possibly with retrieval at each step), and a replanner revises the remaining steps given the results. **Reflection** loops add a critic that reviews a draft answer against the evidence and the question and requests improvements. **Multi-agent** designs split responsibilities (for example, a supervisor routing to specialist agents for documents, data and graph queries). They improve modularity and let each agent have a short, focused prompt and toolset, but they add coordination cost and make failures harder to trace. Start with a single agent and split only when prompts become unwieldy.

4.4 Memory

Agents need two kinds of memory. **Short-term memory** is the state of the current conversation or task, persisted with a checkpointer so that a thread can be resumed and inspected. **Long-term memory** stores facts and preferences across sessions in a separate store, typically retrieved like any other knowledge source. Be selective about what you store, give users control over it, and never let the agent store unverified claims as facts.

5. Guardrails for Loops and Autonomy

Autonomy introduces failure modes that pipelines do not have. An agent can loop indefinitely, call tools with harmful inputs, burn the budget on a hopeless query or follow instructions that were planted in a retrieved document. Guardrails must be designed in from the start, and they should live in code, not only in prompts.

Risk	Mitigation
Infinite or very long loops	Hard step limit (recursion limit), attempt counters in state, wall-clock timeout
Runaway cost	Per-request token and tool-call budgets; cheaper models for grading and routing; caching
Repeated identical tool calls	Detect duplicates; return the cached result with a note; stop after N repeats
Harmful tool use (SQL, file, email)	Least-privilege credentials, read-only by default, allow-lists, argument validation, confirmation for side effects
Prompt injection through retrieved content	Treat tool output as data; strip instructions; isolate untrusted text; restrict tools reachable after reading untrusted content
Confident wrong answers	Grounding checks, citations required, abstention when evidence is weak
Opaque failures	Trace every step: prompts, tool calls, results, state diffs; replayable runs
Irreversible actions	Human-in-the-loop interrupts before side-effecting steps

Table 2. Agentic failure modes and corresponding controls.

Human-in-the-loop is a first-class feature in graph-based frameworks: the graph can pause before a node, persist its state, wait for a human to approve or edit, and then continue. For tasks with consequences (sending messages, changing records, spending money) this should be the default rather than the exception.

6. Knowledge Graphs and GraphRAG

Vector retrieval treats a corpus as a bag of independent passages. Many questions, however, are about **relationships**: "Which suppliers are affected if factory F closes?", "How are these two research groups connected?", or "What are the main themes across all these reports?". The answers are scattered across many passages, linked by entities that no single chunk fully describes. Graph-based retrieval represents entities as nodes and relationships as edges, making these connections explicit and traversable.

6.1 Two kinds of graph-based RAG

- **Querying an existing knowledge graph.** The data already lives in a graph database (for example, Neo4j). The system translates the question into a graph query language (Cypher) with a Text-to-Cypher step, executes it, and verbalises the result. This mirrors Text-to-SQL.
- **Building a graph from unstructured text.** An LLM extracts entities and relations from documents, the graph is constructed automatically, and retrieval combines vector search with graph traversal. **Microsoft GraphRAG** is the best-known example.

6.2 The GraphRAG indexing pipeline

GraphRAG (Edge et al., 2024, *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*) targets questions that require a **global** understanding of a corpus, which conventional vector RAG handles poorly. Its indexing pipeline has these stages:

1. **Chunk** the documents into text units.
2. **Extract** entities, relationships and key claims from each unit with an LLM.
3. **Build** a graph in which entities are nodes and relationships are weighted edges (weights accumulate when the same relation is found repeatedly).
4. **Detect communities**: cluster the graph into hierarchical groups of densely connected entities using the Leiden algorithm.
5. **Summarise** each community into a report with an LLM, at each level of the hierarchy. These reports are a compact, multi-level description of the corpus.

6.3 Local and global search

At query time GraphRAG offers two modes. **Local search** answers questions about specific entities: it finds entities related to the query (by embedding), expands to their neighbours, relationships, source text units and relevant community reports, and builds a context from these. **Global search** answers corpus-wide questions: it takes community reports, asks the model to produce a partial answer from each batch (the **map** step), and then combines the partial answers into a final response (the **reduce** step). This is what makes questions like "What are the main themes?" answerable at all.

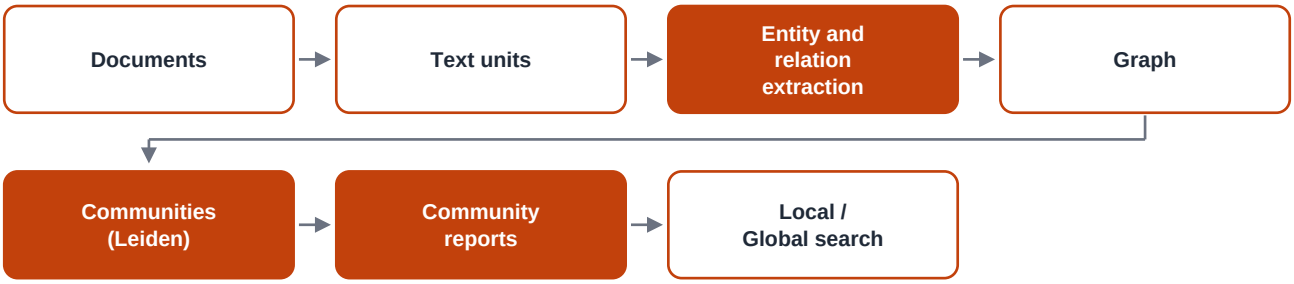


Figure 3. The GraphRAG pipeline. Indexing is LLM-intensive; query-time modes serve different question types.

6.4 A minimal graph construction sketch

The sketch below shows the core idea on a small scale: extract triples with structured output, build a NetworkX graph, find communities and prepare them for summarisation. Production systems add entity resolution (merging "Acme Corp" and "Acme Corporation"), confidence scores, provenance links back to source chunks, and incremental updates.

```
import networkx as nx
from pydantic import BaseModel

class Triple(BaseModel):
    subject: str
    relation: str
    object: str

class Triples(BaseModel):
    triples: list[Triple]

extractor = llm.with_structured_output(Triples)

G = nx.Graph()
for chunk in chunks:
    out = extractor.invoke(
        "Extract (subject, relation, object) facts about named entities from the text. "
        f"Use canonical names.\n\n{chunk.page_content}")
    for t in out.triples:
        s_, o_ = t.subject.strip().lower(), t.object.strip().lower()
        if G.has_edge(s_, o_):
            G[s_][o_]["weight"] += 1
        else:
            G.add_edge(s_, o_, relation=t.relation, weight=1, source=chunk.metadata["chunk_id"])

communities = nx.algorithms.community.louvain_communities(G, weight="weight", seed=42)
for i, members in enumerate(communities):
    facts = [f"{u} -[{G[u][v]['relation']}]-> {v}" for u in members for v in G[u] if v in members]
    # next: summarise `facts` with the LLM into a community report and embed the report
```

Listing 4. Triple extraction and community detection (Louvain here; GraphRAG uses Leiden).

6.5 Text-to-Cypher and graph-enhanced vector search

When a graph already exists, two retrieval patterns are common. **Text-to-Cypher** generates a read-only Cypher query from the question and the graph schema, exactly like Text-to-SQL, and needs the same validation and sandboxing. **Graph-enhanced vector search** first finds seed nodes by vector similarity over node descriptions or chunk text, then expands along relationships to collect connected context, which is then reranked and passed to the model. A typical neighbourhood-expansion query is shown below.

```
// Seed entities come from vector search; expand up to two hops, bounded and read-only.
MATCH (e:Entity)
WHERE e.id IN $seed_ids
MATCH path = (e)-[*1..2]-(n:Entity)
RETURN e.name AS seed, [r IN relationships(path) | type(r)] AS rels, n.name AS neighbour,
       n.description AS description
LIMIT 60
```

Listing 5. Bounded two-hop expansion around seed entities in Cypher.

6.6 Alternatives and trade-offs

Full GraphRAG indexing is expensive because every chunk passes through an LLM, and it must be partially recomputed when the corpus changes. Lighter variants exist: **LightRAG** builds a simpler entity-relation graph with dual-level (specific and abstract) retrieval and supports incremental updates; **HippoRAG** applies personalised PageRank over a knowledge graph, inspired by hippocampal memory indexing, to retrieve associatively connected passages. These evolve quickly, so evaluate current implementations against your own questions.

Question type	Vector RAG	Graph-based RAG
Single-fact lookup	Excellent	Unnecessary overhead
Multi-hop relationship (A is linked to B, B to C)	Weak: needs several lucky retrievals	Strong: traversal follows links

Question type	Vector RAG	Graph-based RAG
Corpus-wide themes and summaries	Poor: no chunk holds the answer	Strong: community reports (global search)
Frequently updated documents	Easy incremental updates	Harder: graph and summaries need refresh
Indexing cost	Embeddings only	LLM calls on every chunk, plus summaries
Explainability	Passages as evidence	Paths and entities as evidence

Table 3. When to prefer each approach.

> **TIP** Do not replace vector retrieval with a graph; add the graph where relationships matter. A hybrid system that routes relationship and global questions to the graph and lookup questions to vector search gets the best of both.

7. Choosing the Right Architecture

The following guide maps requirements to architecture. Begin at the top and move down only when evidence from evaluation demands it.

Situation	Suggested architecture	Why
Mostly factual lookups over one corpus; latency matters	Advanced pipeline (Parts 1-3)	Predictable, fast, cheap, easy to test
Variable quality of retrieval; users ask outside the corpus	Pipeline plus CRAG-style grading and fallback	Quality gate with bounded extra cost
Mixed easy and hard questions	Adaptive routing by complexity	Spend iteration only where needed
Several heterogeneous sources (docs, SQL, graph, web)	Tool-using agent or router plus specialised chains	Model chooses sources and combines results
Multi-hop reasoning and comparison across entities	Decomposition, iterative retrieval, or graph traversal	Evidence must be gathered stepwise
Corpus-level synthesis (themes, trends)	GraphRAG global search or hierarchical summaries (RAPTOR)	No single chunk contains the answer
Actions with side effects	Agent with human-in-the-loop approvals	Safety and accountability

Table 4. Matching problem characteristics to architecture.

7.1 Case study: an enterprise support assistant

Consider an assistant for a hardware company's support team. Questions fall into four types: troubleshooting (answered by manuals), entitlement (SQL: is this serial number under warranty?), relationships (which components are shared between product lines, answered from a parts graph) and policy (documents with frequent updates). A pure pipeline would fail on entitlement and relationship questions. A fully autonomous agent would be unnecessarily slow and unpredictable for the common troubleshooting case.

A pragmatic design is a **router-first hybrid**. A fast classifier routes each query: troubleshooting and policy go to the hybrid-retrieval pipeline with a CRAG-style grader; entitlement goes to Text-to-SQL; relationship questions go to the graph; ambiguous or multi-part questions go to an agent that can call all three tools under a step budget and ask the user a clarifying question when needed. Each path has its own evaluation set, latency target and fallback. This keeps the median path cheap while giving the hard cases the extra capability

they need.

7.2 Evaluating agentic systems

Agentic systems need evaluation beyond the final answer. Part 5 treats this in depth, but the essentials are: measure **task success** end to end; inspect the **trajectory** (were the right tools called, with sensible arguments, in a reasonable number of steps?); track **cost and latency** distributions, particularly the tail; and run **regression suites** because small prompt changes can alter behaviour in unexpected places.

7.3 Checklist

- Start from the simplest pipeline that meets quality; add reflection only for measured failure classes.
- Implement grading and fallback as explicit graph nodes with counters, not as hidden prompt behaviour.
- Give every tool a precise description, typed inputs, bounded outputs and least-privilege access.
- Set step, token and time budgets, and make the system end gracefully when they are exhausted.
- Treat all retrieved and tool-returned text as untrusted data.
- Use graph retrieval for relationship and corpus-level questions, not as a replacement for vector search.
- Trace every run with enough detail to replay and debug it.
- Require human approval for any action with side effects.

7.4 Summary and further reading

Agentic and graph-based RAG extend the pipeline with judgement and structure. Reflection patterns (Self-RAG, CRAG, Adaptive RAG, FLARE) add the ability to notice poor evidence and react; LangGraph-style graphs make that logic explicit, testable and bounded; tool-using agents handle heterogeneous sources and multi-step tasks; knowledge graphs and GraphRAG answer questions about relationships and whole corpora. The price is complexity, cost and variance, which is why guardrails, budgets, tracing and evaluation are inseparable from these designs.

Key references: Asai et al., *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection* (2023); Yan et al., *Corrective Retrieval Augmented Generation* (2024); Jeong et al., *Adaptive-RAG* (2024); Jiang et al., *Active Retrieval Augmented Generation* (FLARE, 2023); Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models* (2022); Edge et al., *From Local to Global: A Graph RAG Approach to Query-Focused Summarization* (2024); Guo et al., *LightRAG* (2024); Gutierrez et al., *HippoRAG* (2024).

Next in the series: Part 5, *Evaluation, Observability and Production*, which shows how to measure and operate everything built so far.